

pch2csd: From Command-Line Converter to Web-Based Modular Synthesis Environment

Gleb Rogozinski^{1,2}

¹ Omni Audio Oy

² Herzen University

gleb.rogozinski@gmail.com

Abstract. The pch2csd project, initiated in 2015, preserves the sonic heritage of the Clavia Nord Modular G2 synthesizer by converting its patches into Csound code. This paper presents the evolution of pch2csd into a web-based modular synthesis environment that combines the proven Python conversion engine with a real-time visual patching interface. We detail the system architecture, the reuse of the existing codebase through a Flask API, and the design decisions enabling seamless interaction between the visual patch editor and the Csound audio engine. The result is a platform that not only preserves the Nord Modular heritage but extends it into browser-based sound design, allowing users to load, visually edit, and hear patches directly in a web browser without proprietary software.

Keywords: Modular Synthesis Systems, Csound in Web, Csound GUI, Hardware Simulation.

1 Introduction

The Clavia Nord Modular G2, discontinued in 2009, left behind a vibrant community and a vast archive of patches. The pch2csd project began as an effort to liberate these patches from proprietary formats, translating them into Csound [1]. The initial implementation was a command-line tool in Python that converted patches to `.csd` files.

Over time, the converter evolved into a comprehensive engine capable of handling the majority of Nord Modular G2 modules. Each Nord module is mapped to a corresponding Csound User-Defined Opcode (UDO), with parameter mapping tables and connection management handled systematically. The development was informed by close study of the Nord Modular’s characteristics [2].

However, the tool’s utility was limited by its non-interactive nature. Users could convert patches, but could not see the patch structure, modify parameters in real-time, or build new patches visually. This paper describes a comprehensive re-imagining of pch2csd as a web-based application with three goals: to preserve the Nord Modular G2 patch archive through a modern interface; to provide an interactive platform for learning modular synthesis and Csound; and to create a hybrid environment combining visual patching with the power of Csound, enabling a new generation of musicians to explore synthesis through the extensive archive of existing patches.

2 The Original Architecture: pch2csd as a Python Engine

The original pch2csd project implements a straightforward pipeline in Python. The first stage is **file parsing**. The parser processes the binary `.pch2` file to extract module definitions, connection lists, and parameter values, according to Michael Dewberry’s guide [3]. This stage handles the format’s quirks, such as “in-to-in” cables which must be resolved into standard output-to-input connections.

The second stage is **module mapping**. Each Nord module is mapped to a corresponding Csound UDO defined in template files that specify the Csound implementation, parameter ranges, and input-output types. The UDO system reads annotations embedded in the Csound code — markers for parameter mapping tables and mode selectors — allowing the converter to validate that the UDO template correctly matches the Nord module’s capabilities.

The third stage is **code generation**. The converter composes a complete Csound `.csd` file with global instrument definitions, function tables for parameter mapping, UDO definitions, and module instantiations. A critical component is the management of Csound’s `zak` space, which provides a flexible audio and control-rate routing matrix. The converter allocates audio-rate and control-rate

buses dynamically, connecting module outputs to inputs through this bus system. When a cable connects two modules of different rates, the converter automatically inserts a rate conversion module, preserving the behavior of the original patch.

3 The Web Architecture

Our new implementation reimagines pch2csd as a full-fledged web application with three layers: the browser frontend, the Flask backend, and the Python conversion engine (see Figure 1).

Flask as the Orchestrator. The Flask application handles file uploads and patch conversion requests. When a user uploads a `.pch2` file, the endpoint invokes the existing Python conversion engine, which parses the file and returns a structured JSON representation of the patch. This JSON format serves as the bridge between the legacy codebase and the modern frontend. Using Flask allows us to encapsulate the complex, stateful logic of the Python converter in a separate process, ensuring the browser frontend remains lightweight and responsive.

Visual Patching with Canvas. The frontend is a rich JavaScript application built around a component-based architecture, with all rendering performed on an HTML Canvas element. At its core is a visual patch editor where modules are rendered as draggable panels with input and output jacks. Connections between modules are drawn as Bézier curves with colors indicating signal type: red for audio, blue for control, yellow for logic (see Figure 2).

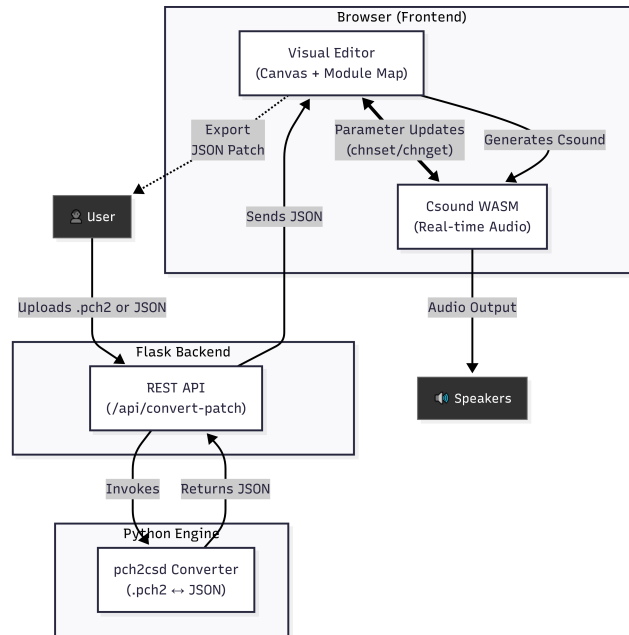


Fig. 1. System architecture showing the three-layer structure: browser frontend with visual patch editor and Csound WebAssembly engine, Flask backend for API orchestration, and Python engine for patch conversion.

The entire visual system is driven by JSON descriptions of modules. Each module is defined by a JavaScript file exporting a structured object containing metadata and visual components — sliders, knobs, dropdown menus, text labels, input and output jacks, LEDs, lines, etc — each with positions and properties. The module definition includes a type identifier linking it to the corresponding Csound UDO file. This data-driven approach means the rendering engine iterates over the components array and instantiates the appropriate primitives, making the system highly extensible.

To populate the system with module descriptions, we developed a converter that transforms the module descriptions from the NMG2 Open Source Editor project [4] into our JSON format. The original descriptions are stored as Delphi component definitions in plain text files using a custom markup language — each module is a declarative description of its visual components, their positions,

and their parameters. Our converter parses these files, maps original component types to our JavaScript component classes, and generates the JSON descriptions the frontend expects.

The frontend integrates with Csound through the WebAssembly build, enabling real-time synthesis directly in the browser using the Web Audio API [5]. When the structure of a patch changes — adding or removing modules, creating or deleting connections — the frontend generates updated Csound code and sends it to the Csound engine for recompilation. For parameter adjustments, however, the communication is more lightweight: values are sent via software channels (`chnget/chnset` in Csound), allowing responsive, low-latency control without recompilation. The architecture also supports a bidirectional relationship: the Csound engine can send control data back to the UI, enabling visual feedback for modulation and other dynamic behaviors.

4 System Challenges and Design Decisions

State Synchronization. With three separate layers — the visual patch editor, the internal module map, and the Csound engine — maintaining consistency required a single source of truth: the module map. When a module is added, removed, or moved, the map is updated first, and changes propagate to both the visual renderer and the Csound code generator. This unidirectional data flow simplifies debugging and ensures all components remain in sync, particularly important with cable connections involving multiple modules and jack indices.

Performance Optimization. To maintain smooth interaction without excessive CPU usage, we implemented a "dirty flag" system for the rendering loop. The canvas is only redrawn when the internal state changes, with the grid background cached as an image rather than recalculated each frame. For animated elements like VU meters and oscilloscopes, updates are throttled to a lower frame rate. This approach reduces CPU load significantly during idle periods while maintaining responsiveness during interaction.

Cable Color System. In the Nord Modular, cables carry semantic meaning through their colors: red for audio signals at 96 kHz, blue for control signals at 24 kHz, yellow for logic signals at 24 kHz, and orange for logic signals at 96 kHz. Additionally, a white cable indicates a special case — a connection between two outputs, which in the original system represents the absence of a normal input. Our implementation maintains both functional color, determined by the type of signal being routed, and visual color, which the user can customize. By default, the visual color matches the functional color, but users can override it through a color picker accessed via the cable's context menu, updating all cables connected to the selected jack simultaneously.

Asynchronous Module Loading. Unlike the command-line version where all definitions load at startup, modules are fetched dynamically when needed through JavaScript's dynamic import. This lazy loading reduces initial load time and supports an extensible architecture where users can add new module types without restarting the application. The system currently supports over 100 Nord G2 modules, with ongoing testing of the converter against a library of community patches.

Memory Management. When a module is removed, all connected cables are deleted first, then the module is removed from the layer manager and components array, and the Csound generator removes the module from its internal state. If no modules of that type remain, the corresponding UDO include directive is removed from the generated code, preventing memory leaks and keeping the Csound engine state clean.

5 Future Work and Conclusions

The pch2csd project has evolved from a preservation tool into a comprehensive web-based modular synthesis environment. By reusing the proven Python engine through a Flask API and building a rich JavaScript frontend with Canvas-based rendering, data-driven module descriptions, and real-time Csound synthesis, we have created a platform that preserves the Nord Modular heritage while extending it into a new era of browser-based sound design.

What began as a focused effort to convert Nord Modular patches has revealed a much larger possibility. The modular system we have built — with its JSON module descriptions, Canvas rendering engine, and Csound integration — is not merely a tool for a single synthesizer. It is a framework with the potential to support modular environments around Csound UDO collections, as suggested by this initial implementation. The same architecture that renders a Nord Modular patch can render a

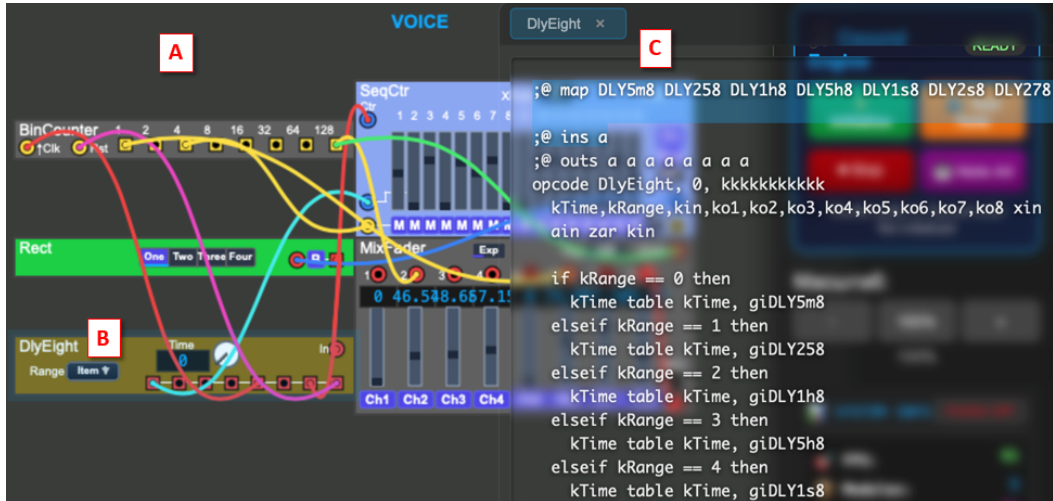


Fig. 2. The web-based modular synthesis interface. (A) Visual patch editor workspace. (B) Selected module with highlighted controls and jacks. (C) Csound code editor showing the module’s UDO implementation. The displayed test patch validates connection integrity; cable colors indicate signal types (audio, control, logic) with user-defined overrides.

library of classic synthesizer modules, a collection of DSP building blocks, or an educational toolkit for teaching sound synthesis. In essence, pch2csd can serve as a foundation for a new graphical user interface for Csound itself — one that is visual, modular, and web-native. By providing a visual patching layer over Csound’s powerful audio engine, we are creating a bridge between the immediacy of modular thinking and the depth of Csound’s synthesis capabilities.

Several directions for future development are already taking shape. Live parameter modulation through control voltage routing will allow any control signal to modulate any parameter, mirroring the Nord Modular’s patching philosophy. The G2 was chosen as the initial target because its official demo version, still available on Clavia’s website, enabled validation of the conversion accuracy without requiring physical hardware, making the project accessible from the outset. We have since acquired a Nord Modular G1 and plan to extend support to it, given its simpler, openly documented file format. A comparative evaluation between the original hardware and the web application is planned as future work. Integration with existing Nord Modular patch editors would allow users to edit patches in their preferred environment and hear results immediately in the browser. The web platform also opens possibilities for collaboration — sharing patches as URLs, creating libraries of user-contributed modules, and real-time collaborative patching sessions. These developments would build on the community spirit that has kept the Nord Modular alive long after its discontinuation.

We invite contributions from the community at <https://github.com/gleb812/csmodular>.

References

1. Rogoziński, G., Chesnokov, M., Cherny, E. “pch2csd: an application for converting Nord Modular G2 patches into Csound code”, Proc. 14th Sound and Music Computing Conf, Espoo, Finland, 2017. 10.48550/arXiv.1707.01653
2. Rogoziński, G., Cherny, E., Chesnokov, M. “Working with pch2csd – Clavia NM G2 to Csound Converter”, Proc. Int’l Csound Conference 2017, Montevideo, Uruguay, 2017. https://csound.com/icsc2017/proceedings/ICSC2017_paper_Rogozinsky.pdf
3. Dewberry, M. “Nord Modular G2 Patch Format.” <http://dewb.org/g2/pch2format.html>, 2005.
4. “NMG2 Open source editor.” <https://sourceforge.net/projects/nmg2editor/>.
5. Lazzarini, V. et al. “Csound on the Web”, Linux Audio Conference, 2014. <https://lac.linuxaudio.org/2014/papers/23.pdf>